

Ce qui rend R unique

5 fonctionnalités qui *séduisent* un développeur Python

RÉSUMÉ

R possède des particularités qui peuvent surprendre quand on vient de Python, mais qui rendent le langage particulièrement puissant, notamment de par sa **flexibilité**.

Ce poster présente **5 mécanismes** qui changent la manière d’écrire du code : des opérateurs lisibles, du dispatch léger, des expressions que l’on peut capturer, un système objet simple et des arguments évalués seulement quand ils sont utilisés.

LES 5 MÉCANISMES

01 Créer ses opérateurs

Nommer une intention directement dans la syntaxe.

R

```
`%within%` <- function(x, range) {  
  x >= range[1] & x <= range[2]  
}  
  
5 %within% c(1, 10) # TRUE  
12 %within% c(1, 10) # FALSE
```

Python

```
class Infix:  
    def __init__(self, f):  
        self.f = f  
    def __ror__(self, x):  
        return Infix(lambda y: self.f(x, y))  
    def __or__(self, y):  
        return self.f(y)  
  
within = Infix(lambda x, r: r[0] <= x <= r[1])  
  
5 |within| (1, 10) # True
```

USAGE

Rendre le code plus proche du vocabulaire métier et du langage naturel.

COMMENT ÇA MARCHE

Un opérateur infix est une fonction dont le nom est entouré par des %.

02 Surcharger des opérateurs

Adapter une syntaxe connue à vos propres objets.

R

```
`+.vec2` <- function(a, b) {  
  vec2(a$x + b$x, a$y + b$y)  
}  
  
p1 + p2
```

Python

```
class Vec2:  
    def __add__(self, other):  
        return Vec2(  
            self.x + other.x,  
            self.y + other.y,  
        )  
  
p1 + p2
```

USAGE

Garder une API naturelle pour des objets spécialisés.

COMMENT ÇA MARCHE

R cherche une méthode S3 nommée comme l'opérateur et la classe.

03 Capturer une expression

Traiter du code comme une donnée manipulable.

R

```
label <- function(expr) {  
  deparse(substitute(expr))  
}  
  
label(log(x + 1))
```

Python

```
import ast  
  
def label(expr):  
    tree = ast.parse(expr, mode='eval')  
    return ast.unparse(tree)  
  
label('log(x + 1)')
```

USAGE

Crérer des messages, formules, message de debug ou pipelines plus lisibles.

COMMENT ÇA MARCHE

`substitute()` récupère l'expression **avant** son évaluation.

04 Utiliser S3 simplement

Faire de l'orienté objet sans cérémonie.

R

```
describe <- function(x) {  
  UseMethod('describe')  
}  
  
describe.lm <- function(x) {  
  'un modèle linéaire'  
}
```

Python

```
from functools import singledispatch  
  
@singledispatch  
def describe(x):  
    raise NotImplementedError  
  
@describe.register  
def _(x: LinearModel):  
    return 'un modèle linéaire'
```

USAGE

Écrire une fonction générique puis spécialiser son comportement.

COMMENT ÇA MARCHE

Le dispatch appelle `describe.<classe>()` selon la classe de l'objet.

05 Profiter de la lazy evaluation

Définir des valeurs par défaut qui dépendent des autres arguments.

R

```
scale2 <- function(x, mu = mean(x), sigma =  
sd(x)) {  
  (x - mu) / sigma  
}
```

Python

```
def scale2(x, mu = None, sigma = None):  
    if mu is None:  
        mu = mean(x)  
    if sigma is None:  
        sigma = sd(x)  
  
    return (x - mu) / sigma
```

USAGE

Offrir des defaults intelligents sans demander plus à l'utilisateur.

COMMENT ÇA MARCHE

Les arguments sont des promesses : ils sont évalués seulement si nécessaire.

PDF EN LIGNE



SCAN ME